

Задание 4. Повышение наблюдаемости контейнерной системы

1. Постановка задачи

Приложение работает в контейнерах на нескольких узлах. Периодически часть контейнеров перезапускается, а время ответа системы заметно возрастает. Команда чаще всего узнает о проблеме не по своим инструментам, а по сообщениям пользователей. Логи приложений хранятся локально внутри контейнеров, поэтому после перезапуска часть информации теряется или становится труднодоступной. При этом у команды нет единой картины по метрикам, логам и признакам деградации системы.

Предположим, что нужно повысить наблюдаемость системы и сделать разбор инцидентов быстрее и надежнее.

Задание: предложите, какие инструменты и практики из модуля следует добавить в такую архитектуру, и поясните, какую именно проблему решает каждый предложенный элемент. Достаточно концептуального ответа без написания каких-либо конфигурационных файлов.

2. Краткий анализ исходной системы

Ключевая проблема архитектуры — недостаточная observability. Приложение работает в контейнерах на нескольких узлах, но команда не имеет единой картины по состоянию сервисов, контейнеров, узлов, задержек и ошибок.

Инциденты обнаруживаются поздно: рост времени ответа и перезапуски контейнеров становятся заметны в основном по жалобам пользователей. Диагностика осложняется тем, что логи хранятся локально внутри контейнеров и после перезапуска частично теряются или становятся труднодоступными.

Главный архитектурный дефицит: нет централизованного мониторинга, централизованного логирования, визуализации и автоматического оповещения о деградации.

3. Что нужно добавить

Целевая схема должна объединить метрики, логи, дашборды и алерты. Тогда команда сможет видеть деградацию заранее, сохранять данные после перезапусков и быстрее находить причину инцидента. Метрики и логи должны храниться вне контейнеров с заданным сроком хранения, чтобы разбор был возможен не только в момент сбоя, но и после него.

Направление	Инструмент	Что собирать / делать	Какую проблему решает
Метрики сервисов	Prometheus	Latency p95/p99, RPS, 4xx/5xx, доступность, health-check, технические метрики приложения.	Показывает деградацию по данным, а не по жалобам. Дает основу для истории и алертов.
Метрики контейнеров	cAdvisor	CPU, RAM, сеть, filesystem, лимиты ресурсов, restart count, признаки OOMKilled.	Позволяет увидеть перегруженные или часто перезапускающиеся контейнеры и понять, связана ли проблема с ресурсами контейнера.

Метрики узлов	Node Exporter	CPU, RAM, disk, network, load average, filesystem и состояние физических или виртуальных узлов.	Помогает понять, деградация связана с конкретным контейнером или с самим сервером/узлом.
Логи	Promtail + Loki	Сбор логов из stdout/stderr контейнеров, хранение вне контейнеров, поиск по service, container, node, level, request_id и времени.	Логи не теряются после перезапуска и доступны в едином месте для анализа инцидентов.
Хранение истории	Retention в Prometheus/Loki	Заданный срок хранения метрик и логов, например 7–30 дней, вне жизненного цикла контейнеров.	Позволяет анализировать инциденты после перезапусков и сравнивать текущее состояние с историей.
Визуализация	Grafana	Дашборды по Prometheus и Loki: задержки, ошибки, рестарты, нагрузка, состояние узлов и сервисов.	Дает единую картину системы и ускоряет поиск проблемного сервиса, контейнера или узла.
Реагирование	AlertManager	Уведомления при росте latency, 5xx, частых рестартах, нехватке памяти, заполнении диска, падении health/readiness-check.	Команда узнает о проблеме автоматически до массовых жалоб пользователей.

4. Признаки деградации, которые нужно отслеживать

• Рост p95/p99 времени ответа	• Рост доли HTTP 5xx
• Частые рестарты контейнеров	• OOMKilled или память близко к лимиту
• Высокая загрузка CPU контейнера или узла	• Заполнение диска или проблемы filesystem
• Провал health/readiness-check	• Ошибки подключения к БД или внешним сервисам
• Перегрузка сети или узла	• Рост количества timeout-ошибок

5. Практики эксплуатации

- Liveness-check — показывает, что процесс жив и контейнер не нужно перезапускать. Readiness-check — показывает, что сервис готов принимать трафик. Health-check — отражает общий статус сервиса и его зависимостей.
- Структурированные логи — JSON-логи с полями timestamp, level, service, endpoint, status, duration, error. Основные логи приложения должны писаться в stdout/stderr контейнера, а не только в локальные файлы внутри контейнера.
- Request ID / Trace ID — Request ID помогает найти все логи по одному запросу, а Trace ID нужен, если запрос проходит через несколько сервисов. Для полноценной трассировки можно добавить OpenTelemetry + Tempo или Jaeger.
- SLI/SLO — фиксируются целевые показатели: latency, error rate, availability. По ним задаются пороги для мониторинга и алертов.
- Runbook для инцидентов — для типовых алертов описано, какие дашборды и логи проверять, какие команды выполнять и кого уведомлять.
- Единые labels в метриках и логах — service, instance, container, node, environment. Это позволяет быстро фильтровать данные по конкретному сервису, контейнеру или узлу.
- Базовые алерты — p95 latency выше нормы 5 минут; HTTP 5xx больше 2–5%; контейнер перезапустился больше 3 раз за 10 минут; RAM выше 85%; диск заполнен больше 80–90%; health/readiness-check падает.
- Postmortem после серьезных инцидентов — фиксируются причина, последствия, действия по устранению и меры предотвращения повторения.

6. Сценарий разбора инцидента

1. Контейнер начинает часто перезапускаться или отвечать медленнее. cAdvisor фиксирует restart count и ресурсные ограничения контейнера, Node Exporter показывает состояние узла, а Prometheus собирает p95/p99 latency, CPU/RAM и ошибки.
2. Срабатывает AlertManager. Команда получает уведомление в рабочий канал до массовых жалоб пользователей.
3. Инженер открывает Grafana. На дашборде видно, когда началась деградация, какой сервис, контейнер или узел затронут и как изменились latency, 5xx, нагрузка и рестарты.
4. Инженер переходит к логам в Loki. По тому же периоду, service, container, node, request_id или trace_id находятся ошибки приложения, контейнера или внешней зависимости.

7. Итог

В архитектуру нужно добавить Prometheus, cAdvisor, Node Exporter, Promtail, Loki, Grafana и AlertManager, а также практики structured logging, request_id/trace_id, health/liveness/readiness checks, SLI/SLO, runbook и postmortem. Решение переводит команду от реактивного режима «узнаем от пользователей» к проактивному мониторингу по метрикам, логам и алертам. В результате команда заранее видит деградацию, не теряет логи после перезапусков и быстрее находит причину инцидента.